



Using Schematron to check XML documents

Introduction to xMCF Team
(Kevin Le Tutour, AFNeT Services)

Agenda

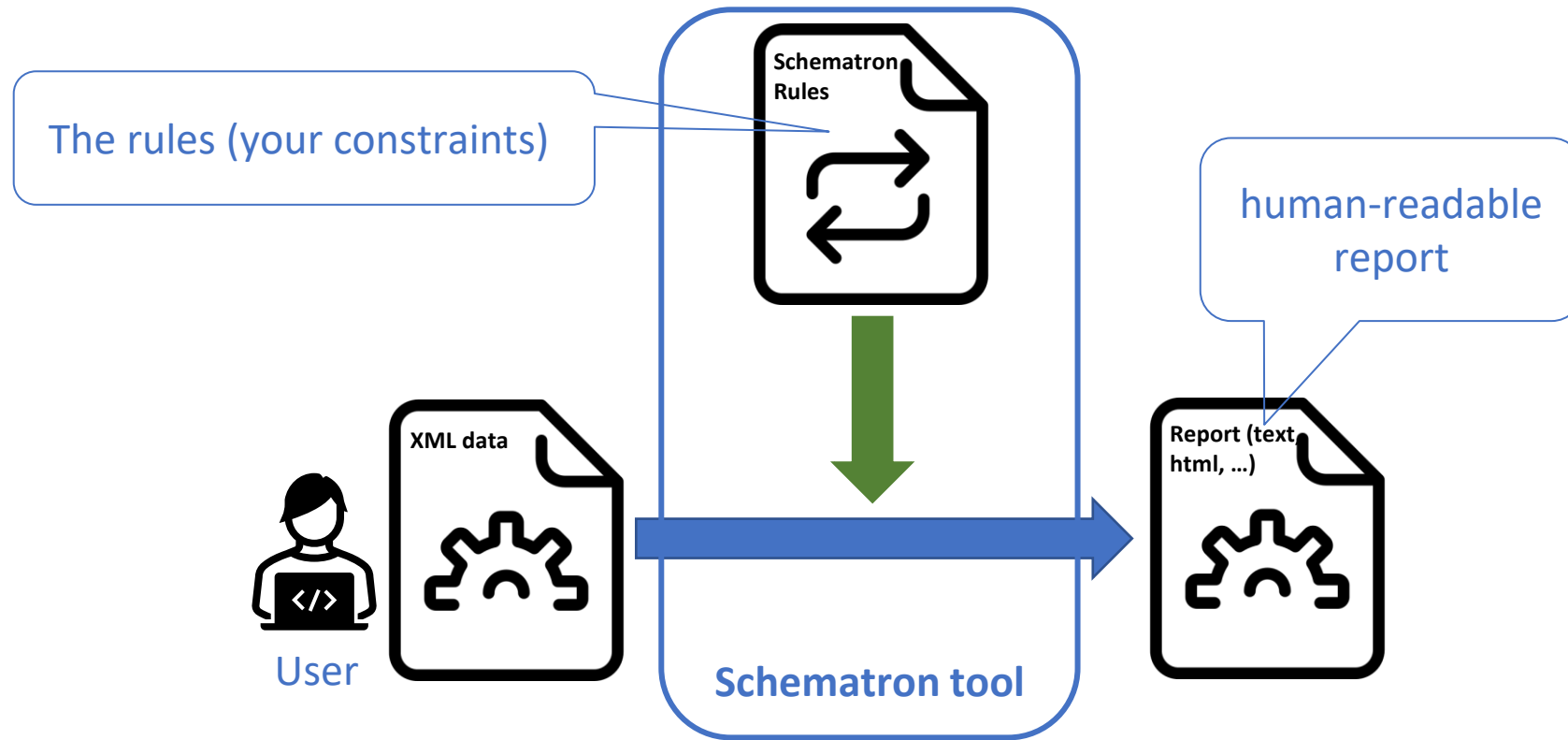
Using Schematron to check XML documents

- Schematron
- STEP AP242
- χ MCF
 - Comparison with XSD:assert



Schematron

Introduction



Schematron

Introduction

SCH rule

```
<sch:schema>
  <sch:pattern>
    <sch:rule context="quantity">
      <sch:assert test=". > 0">
        An order item must have a quantity of at least 1.
      </sch:assert>
    </sch:rule>
  </sch:pattern>
</sch:schema>
```

XML data

```
<order id="101">
  <item>
    <name>Wireless Mouse</name>
    <quantity>0</quantity>
    <price>25.00</price>
  </item>
</order>
```

Text report

Validation Error

Location: /order/item/quantity

Message: An order item must have a quantity of at least 1.

Value found: 0

Schematron

Introduction

- The objective is to automatically verify XML documents against rules **not covered by XSD language**
- Schematron is an XML rule-based validation language
 - The Schematron defines XPATH assertions on the **structure** and **content** of an XML document, that cannot be verified by an XSD check
 - If the assertion fails, diagnostic messages are reported
 - Schematron is an international standard: ISO/IEC 19757-3:2020
- Schematron rules does not replace XSD validation but extend it with more constraint
- A tool is required to
 - Execute Schematron rules files on XML files
 - Run XSL transformations (provided by the standard)
 - Display the messages
 - Text or HTML or other types of reports



Schematron

The language

```

<?xml version="1.0"?>
<sch:schema xmlns:sch="http://purl.oclc.org/dsdl/schematron" title="Rules for [type] documents">

    <sch:pattern name="[your name]">

        <sch:rule context="[your XPATH or XSLT pattern]"> <!-- Rules matchings an element in your document -->

            <sch:let name="variable" value="[XPath]" /> <!-- Setup variable to use it in other XPATH or message -->

            <sch:report test="[your XPATH test (assertion)]"> <!-- Reported as an info if test is true -->
                [your message]
            </sch:report>

            <sch:assert test="[your XPATH test]"> <!-- Failed assertions are reported as errors -->
                [your message ... <sch:value-of select="[XPath]" /> ...] <!-- You can use functions in the message -->
            </sch:assert>
            <!-- Add more <sch:report> or <sch:assert>... -->
        </sch:rule>
        <!-- Add more <sch:rule> ... -->
    </sch:pattern>
    <!-- Add more <sch:pattern> ... -->
</sch:schema>

```

Source: dev.pageseeder.com



Schematron

Example of AP242 PDM-IF rule

```
<sch:pattern id="Template_Document-DocumentType">
  <sch:rule context="DataContainer/Document">
    <sch:let name="doc" value="Name/CharacterString/text()" />
    <sch:let name="dtype" value="lower-case(DocumentTypes/ClassString/text())" />
    <sch:assert
      test="matches($dtype,'catalogue')
      or matches($dtype,'manual')
      or matches($dtype,'specification')
      or matches($dtype,'primary geometry')
      or matches($dtype,'secondary geometry')
      or matches($dtype,'nc data')
      or matches($dtype,'fe data')
      or matches($dtype,'sample data')
      or matches($dtype,'process plan')
      or matches($dtype,'check plan')
      or matches($dtype,'drawing')
      or matches($dtype,'structured product data')">
      The document type of '<sch:value-of select="$doc" />' is '<sch:value-of select="$dtype" />' which is not one of the following:
      catalogue, manual, specification, primary geometry, secondary geometry, nc data, fe data, sample data, process plan, check plan,
      drawing, structured product data.
    </sch:assert>
    <sch:report test="not(DocumentTypes/ClassString) and not(DocumentTypes/Class)">
      The document type of '<sch:value-of select="$doc" />' is not a ClassString, and there is no reference to a Class.
    </sch:report>
  </sch:rule>
</sch:pattern>
```



Schematron

Example of AP242 PDM-IF rule

```
<sch:pattern id="002_Template_Identifier"><!-- 4.6.6 RP PDM-IF PAS – Template Identifier -->
```

```
  <sch:rule context="//Id/Identifier">
```

```
    <sch:let name="id" value="@id"/>
```

```
    <sch:let name="role" value="@idRoleRef"/>
```

```
    <sch:let name="context" value="@idContextRef"/>
```

```
    <sch:let name="class" value="//Class[@uid = $role]/Id/@id"/>
```

```
    <sch:assert test="count(//Id/Identifier[@id=$id and @idRoleRef=$role and @idContextRef=$context]) = 1">
```

The trio identifier/role/context is not unique for id '`<sch:value-of select="$id"/>`', role '`<sch:value-of select="$role"/>`', and context '`<sch:value-of select="$context"/>`'.

```
  </sch:assert>
```

```
  <sch:assert test="matches($role,'identification information') or matches($role,'exchange identification information') or matches($class,'identification information') or matches($class,'exchange identification information')">
```

The idRoleRef should be 'identification information' or 'exchange identification information' or a uidRef reference to a Class which declared Id.id attribute as the idRoleRef.

```
  </sch:assert>
```

```
  <sch:assert test="not(../@id and ../Id/Identifier/@id)">The identifiers are mixed.</sch:assert>
```

```
  <sch:assert test="not(../Id/@id = $id)">It is not recommended to duplicate the identification in the Id.id attribute and in the Identifier.id attribute.
```

Moreover, it is not recommended to use Id.id, it shall only be used in special cases like Organization.Id.

```
  </sch:assert>
```

```
  <sch:assert test="count(../Identifier[@idRoleRef=$role]) = 1 or $class!='exchange identification information'">
```

If an object has multiple identifiers, one and only one should have the role 'exchange identification information'.

```
  </sch:assert>
```

```
</sch:rule>
```

```
</sch:pattern>
```



Schematron

Example of reports

STEP XML Quality Checker

Schematron rules check results

9:49 a.m. 21/3/2024.

STEP XML file: "HarnessExample_Flat.xml "

Schematron check with " DomainModel.sch " :15 error(s)

Filters Active - 0

Collapse All Show All Clear All

Entity uid

_100502

1

_103003

1

_103004

1

_201009

1

_201109

1

_201209

1

_202007

1

_202107

1

Entity name

CrossSectionalGroupShapeElementWithTubularCo...

3

CrossSectionalOccurrenceShapeElement

8

ExternalClassSystem

1

PartTerminal

2

TopologyToGeometryModelAssociation

1

Attribute name

AllowedClasses

1

Definition

8

Items

3

RepresentedGeometry

2

Target

1

Bad Reference uid

_101020

3

_102020

2

_103092

1

_103094

1

_104020

1

_121003

1

_122003

1

_313004

2

Show 10 entries

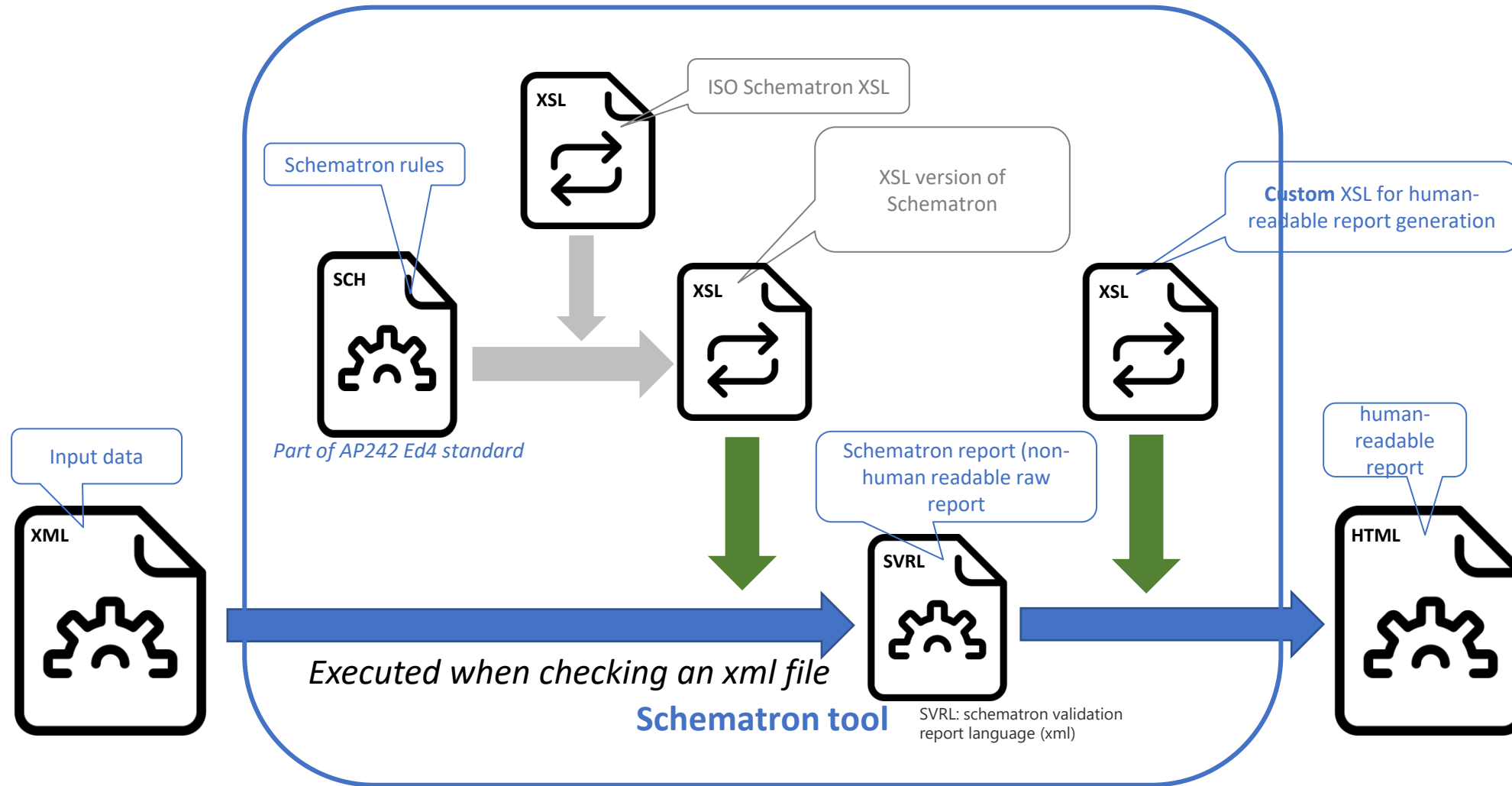
Search:

Schematron file	Entity uid	Entity name	Attribute name	Bad Reference uid	Reference Type	Description	Error Details
DomainModel.sch	_313005	CrossSectionalGroupShapeElementWithTubularCover	Items	_313004	CrossSectionalConstituentElementSelect	(CrossSectionalGroupShapeElementWithTubularCover : _313005) The CrossSectionalGroupShapeElementWithTubularCover.Items contains a bad reference (_313004) must be an element of CrossSectionalConstituentElementSelect.	
DomainModel.sch	_313006	CrossSectionalGroupShapeElementWithTubularCover	Items	_313005	CrossSectionalConstituentElementSelect	(CrossSectionalGroupShapeElementWithTubularCover : _313006) The CrossSectionalGroupShapeElementWithTubularCover.Items contains a bad reference (_313005) must be an element of CrossSectionalConstituentElementSelect.	



Schematron

Script chain (with examples from STEP AP242 projects)

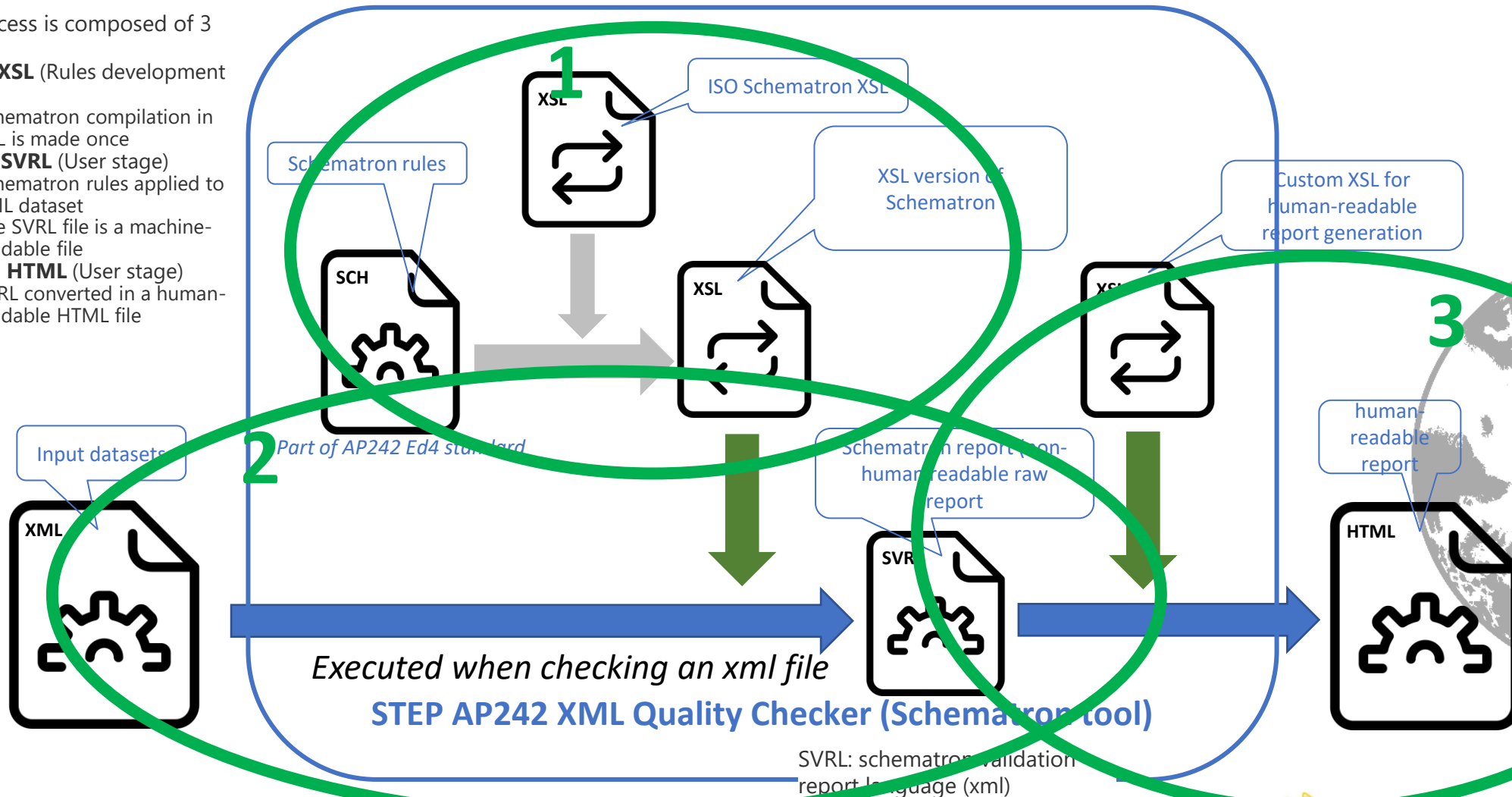


Schematron

Script chain (with examples from STEP AP242 projects)

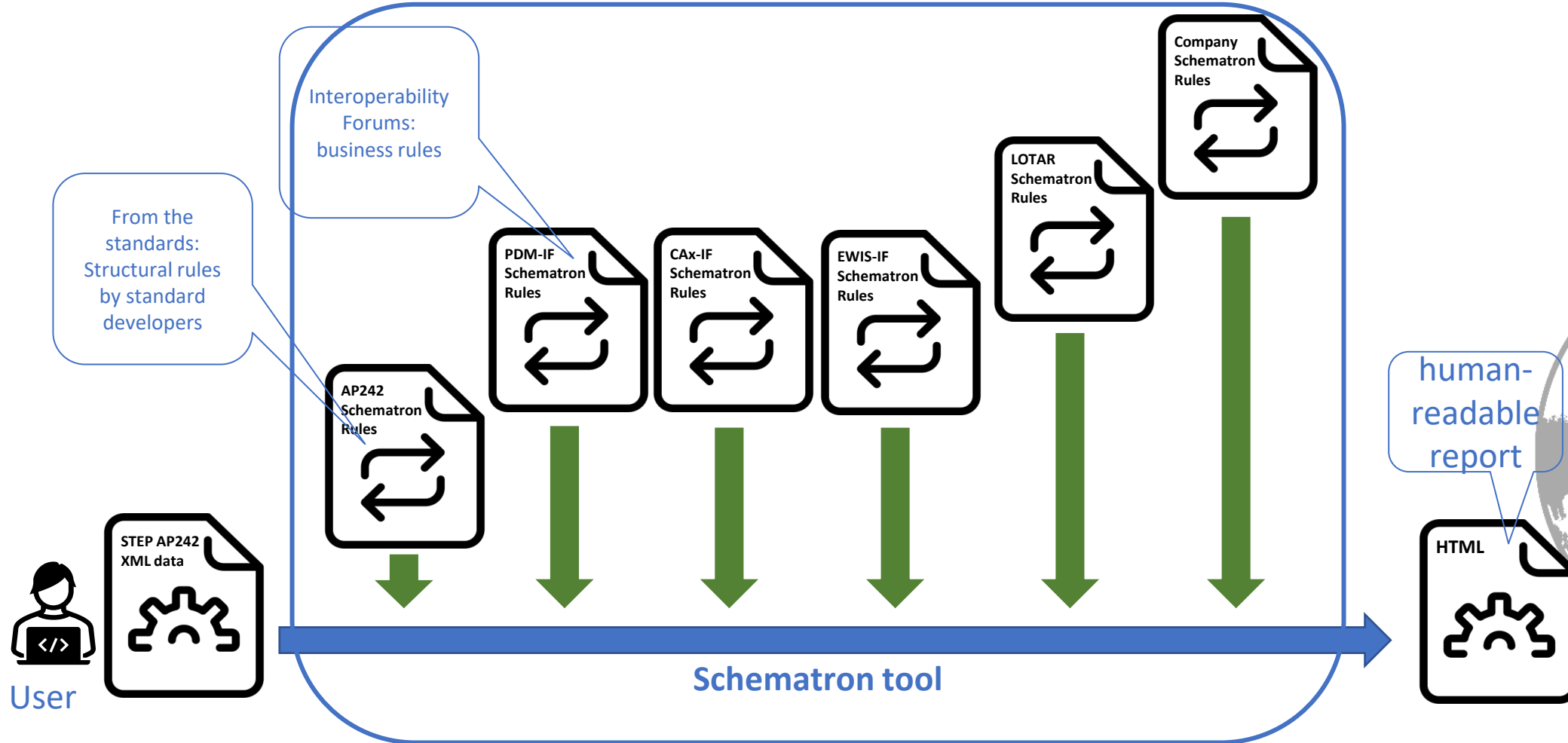
Schematron process is composed of 3 steps:

1. **SCH → XSL** (Rules development stage)
 - Schematron compilation in XSL is made once
2. **XML → SVRL** (User stage)
 - Schematron rules applied to XML dataset
 - The SVRL file is a machine-readable file
3. **SVRL → HTML** (User stage)
 - SVRL converted in a human-readable HTML file



Schematron

Multiple Schematron files



STEP AP242 Standard

Usage of Schematron in STEP AP242 (ISO 10303-242)

Domain model: Managed model-based 3D engineering domain model

ISO/TS 10303-4442:2025(en)
© ISO

Annex C (informative)

Computer interpretable listings

This annex provides the source files associated to this document, including the Domain model XMI in Canonical XMI [31] and UMLDI [30]. It also provides a listing of the Domain model XML schema, its associated Domain model Schematron file and the Domain model EXPRESS file. These listings are available in computer-interpretable form in Table C.1 and can be found at the following URLs:

- XMI, UMLDI, XSD, SCH, EXPRESS:
<http://standards.iso.org/iso/10303/smr/v12/tech/smr/v12.zip>
- XSD: https://standards.iso.org/iso/10303/-4442/ed-5/tech/xml-schema/domain_model/DomainModel.xsd
- SCH: https://standards.iso.org/iso/10303/-4442/ed-5/tech/xml-schema/domain_model/DomainModel.sch
- EXPRESS: https://standards.iso.org/iso/10303/-4442/ed-5/tech/express/domain_model/DomainModel.exp

Table C.1 - Computer interpretable listings

Description	ASCII file	Identifier
AP Domain XMI project file	XMI, UMLDI	ISO/TC 184/SC 4/WG 12 N11492
AP Domain XML schema definition	XSD	ISO/TC 184/SC 4/WG 12 N11493
AP Domain Schematron file	SCH	ISO/TC 184/SC 4/WG 12 N11494
AP Domain EXPRESS schema	EXPRESS	ISO/TC 184/SC 4/WG 12 N11495

© ISO 2025 — All rights reserved

4.2.20.3.20 PartRelationship

This entity is represented in the [ProductDataManagement_BDD](#) diagram.

Entity definition:

A **PartRelationship** is a relationship between two **Part** objects.

SysML specification:

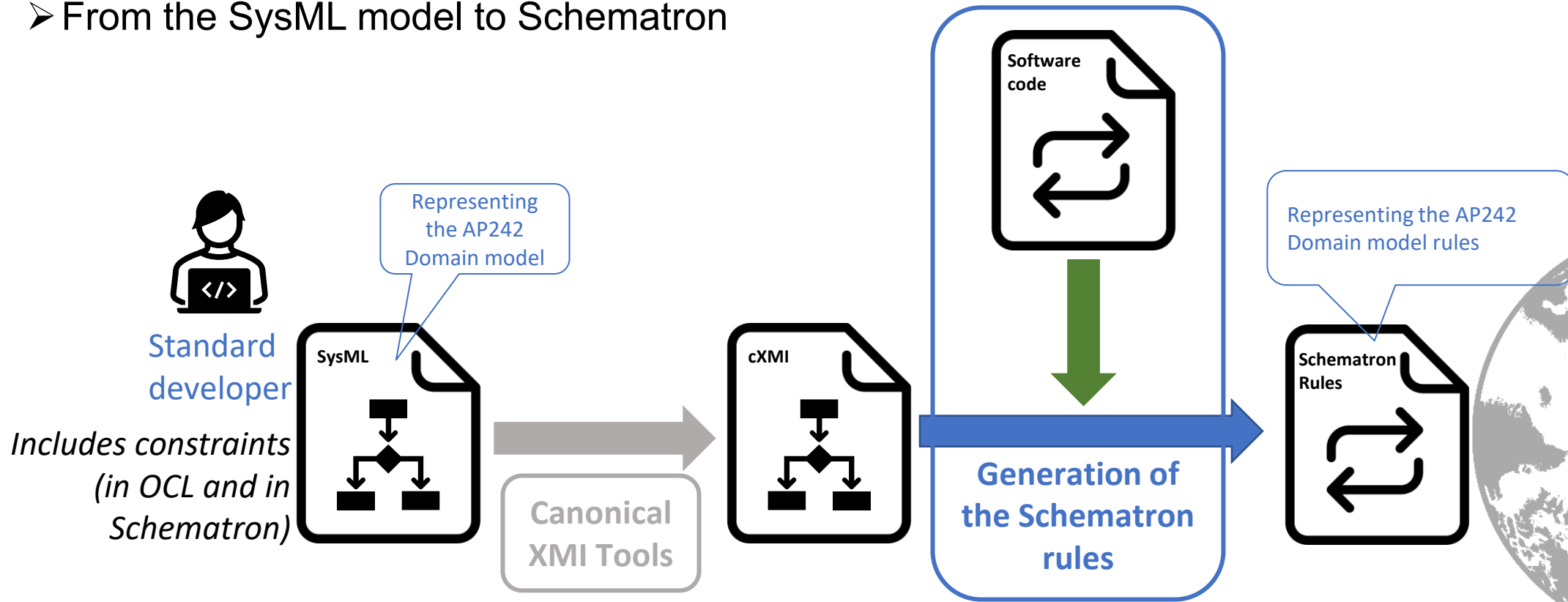
Definition	Cumulative Properties	Mapping	XSD binding
- Subtype: AlternatePartRelationship - Auxiliary: AnalysisAssignmentSelect ApprovalAssignmentSelect CertificationRelationshipObjectSelect - Part Properties: Description : [0:1] of DescriptorSelect Id : [0:1] of IdentifierSelect - Reference Properties: ClassifiedAs : [0:1] of Classification Related : [1] of Part Relating : [1] of Part (attribute represented by an encapsulation in RelationType : [1] of ClassSelect) - Constraints: WR1 (OCL2.0) :: self.Relating <> self.Related			

Definition	Cumulative Properties	Mapping	XSD binding
<pre><xsd:complexType name="PartRelationship"> <xsd:complexContent> <xsd:extension base="BaseObject"> <xsd:sequence> <xsd:element name="ClassifiedAs" minOccurs="0"> <xsd:complexType> <xsd:sequence> <xsd:element name="Classification" type="Reference" maxOccurs="unbounded"> </xsd:sequence> </xsd:complexType> </xsd:element> </xsd:sequence> <xsd:element name="Description" minOccurs="0"> <xsd:complexType> <xsd:group ref="DescriptorSelect"> </pre>			
<pre><sch:pattern id="IndividualPartRelationship_WR1"> <sch:rule context="//IndividualPartRelationship[@uid]"> <sch:let name="entity_name" value="if(@xsi:type) then substring-after(@xsi:type,':') else name()"/> <sch:let name="uid" value="@uid"/> <sch:let name="uidRelating" value="../@uid"/> <sch:let name="uidRelated" value="Related/@uidRef"/> <sch:assert test="not(../@uid = Related/@uidRef)"> The related IndividualPart and the relating IndividualPart shall be different. { - entity_name: <sch:value-of select="\$entity_name"/> -entity_uid:<sch:value-of select="\$uid"/> -rule_name:IndividualPartRelationship.WR1 -relating:<sch:value-of select="\$uidRelating"/> -related:<sch:value-of select="\$uidRelated"/> } </sch:assert> </sch:rule> </sch:pattern></pre>			

STEP AP242 Standard

Schematron development process

➤ From the SysML model to Schematron



From here, all steps are automated and executed once when rules are completely specified

χMCF

Schematron and χMCF Team needs

- Brainstorming on potential requirements from xMCF team
 - χMCF XSD
 - χMCF XML data

```
<?xml version="1.0" encoding="iso-8859-1" standalone="no"?>
<xmcf xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:noNamespaceSchemaLocation="xmcf_3_1_1.xsd">
  <date> 2014-08-07 </date>
  <version> 3.1.1 </version>
  <units length="mm" angle="deg" mass="kg" force="N" time="s"/>
  <connection_group id="1">
    <connected_to>
      <part index="1" label="PART_7000400"/>
      <assy index="23">
        <part label="PART_5000300"/>
        <part label="PART_5000800"/>
      </assy>
    </connected_to>
    <connection_list>
      <connection_1d label="DROP_LINE_11000">
        <sequence_connection_0d spacing="30.0" margin="1.0" priority="spacing">
          <gumdrop diameter="4.0" mass="0.0032" material="CAD_Material"/>
        </sequence_connection_0d>
        <loc_list>
          <loc v="1"> 2169.300 -489.495 1773.936 </loc>
          <loc v="2"> 2165.593 -480.000 1790.221 </loc>
          <loc v="3"> 2165.593 480.000 1790.221 </loc>
          <loc v="4"> 2169.302 489.495 1773.937 </loc>
        </loc_list>
        <appdata>
          <appdata>
        </appdata>
      </connection_1d>
    </connection_list>
  </connection_group>
</xmcf>
```



Xsd:assert with xerces:message

Comparison with Schematron

XML data

```
<order id="101">
  <item>
    <name>Wireless Mouse</name>
    <quantity>0</quantity>
    <price>25.00</price>
  </item>
</order>
```

SCH rule

```
<sch:schema>
  <sch:pattern>
    <sch:rule context="quantity">
      <sch:assert test=". > 0">
        An order item must have a quantity of at least 1.
      </sch:assert>
    </sch:rule>
  </sch:pattern>
</sch:schema>
```

Equivalent XSD assert=

XSD schema

with assert and xerces message

```
<xsd:complexType name="order">
  <xs:element name="quantity" type="xsd:integer"/>
  <xsd:assert test="quantity > 0" xerces:message="Error: An order item must have a quantity of at least 1."/>
</xsd:complexType>
```



XSD with assert and Schematron

Comparison

XSD with assert and xerces message

- **Primary goal**
 - **Data Integrity:** Ensuring the XML document is "well-formed" and mathematically valid according to a schema.
- **Coupling**
 - **Tight:** Assertions are part of the Type definition. If the assertion fails, the element is technically "invalid."
- **Scope**
 - **Local:** Can only "see" the element it is currently inside (and its descendants/attributes).

Schematron

- **Primary goal**
 - **Business Rules:** Validating complex relationships, clinical guidelines, or legal requirements.
- **Coupling**
 - **Loose:** Rules are separate from the structure. You can run different Schematron files against the same XML.
- **Scope**
 - **Global:** Can "jump" anywhere in the document to compare distant nodes using absolute XPath.



XSD with assert and Schematron

Comparison

Constraint Type	Possible in XSD 1.1?	Possible in Schematron?
Sibling-to-Sibling check	No (Limited to descendants)	Yes (Full XPath access)
Check against Root element	No (Unless inside Root)	Yes (Using <code>/</code> or <code>ancestor::</code>)
Conditional Warnings	No (Only Errors)	Yes (Using <code>sch:report</code>)
External Document Lookup	No	Yes (Using <code>document()</code> function)



XSD with assert and Schematron

Comparison

- While xsd:assert is powerful, it is still bound by the "Inheritance and Hierarchy" rules of XML Schema. Schematron, however, is purely based on XPath patterns and doesn't care about the tree structure.
- The most significant constraint possible in Schematron that is impossible (or extremely difficult) in XSD 1.1 xsd:assert is
 - **Global Document Correlation:** In XSD 1.1, an xsd:assert must be defined within a complexType. It can only "see" its own attributes and children. It cannot look "up" the tree or at "sibling" branches that are not descendants.
 - **Schematron Capability:** A rule can say: "If any <Invoice> in the entire document has a total > \$10,000, then the <Header> at the very top of the file must contain a ManagerSignature."
 - **XSD Limitation:** The <Header> and <Invoice> are usually in different branches. XSD cannot reach across these branches because the assertion is scoped to the specific element being validated.
 - **Multi-Phase Validation:** Business rules often change based on the lifecycle of the data (e.g., a "Draft" vs. a "Final" submission).
 - **Schematron Capability:** You can define Phases. You can run "Phase 1" to check basic requirements for a draft, and "Phase 2" for a legal filing. You use the same schema file but toggle which rules are active.
 - **XSD Limitation:** XSD is "all or nothing." If a rule is in the schema, the document must follow it to be considered "valid." There is no native concept of "Validation Stages."
 - **Cross-Document Validation** using document()



XSD and Schematron

- There are many ways to connect both file
 - Technically by declaring Schematron files or rules in XSD
 - In your process, eg:
 - 1) XSD check report
 - 2) SCH check report

